🗄️ CATEGORY 6: State Management & Workspaces

> **Difficulty:** Intermediate → Advanced | **Topics:** 12 | **Terraform Interview Mastery Series**

Table of Contents

1. What State Is, Why It Exists, What's Inside It
2. ⚠ Remote Backends — S3+DynamoDB, GCS, Terraform Cloud
3. State Locking — Mechanism, Failure Scenarios
4. ⚠ `terraform state` Commands
5. ⚠ Splitting Large State — When and How
6. Drift Detection — `plan`, `refresh`, Real-World Handling
7. State File Security — Encryption, Access Control, Secret Exposure
8. `terraform force-unlock` — When and How to Use Safely
9. Workspaces — What They Are, OSS vs TFC
10. ⚠ Workspaces vs Separate State Files — When to Use Which
11. ➕ `terraform.workspace` Interpolation in Code
12. ➕ ⚠ Workspace Anti-Patterns & Limitations

Topic 35: What State Is, Why It Exists, What's Inside It

🔵 What It Is (Simple Terms)

Terraform **state** is a JSON file that acts as Terraform's memory. It maps every resource in your configuration to its real-world counterpart in the cloud — tracking resource IDs, current attribute values, and the dependency relationships between resources.

Without state, Terraform has no way to know that `aws_instance.web` in your config corresponds to `i-0abc123def456` in AWS.

🔵 Why State Exists — The Core Problem It Solves

```
The fundamental problem:
  Terraform config = DESIRED state (what you want)
  Cloud provider   = ACTUAL state  (what exists)

  Terraform needs to compute: desired - actual = changes to make

Without state, Terraform would need to:
  → Query ALL cloud resources on every plan
  → Figure out which ones YOUR config owns
  → Detect what changed
```

```
  → This is slow, expensive, and error-prone

With state, Terraform:
  → Reads state to know which resources it manages and their last-known
attributes
  → Calls provider APIs ONLY for those specific resources (refresh)
  → Computes diff: state vs config → plan
  → After apply: updates state to reflect new reality
```

🔵 What's Inside a State File

```
{
  "version": 4,
  "terraform_version": "1.5.0",
  "serial": 47,
  "lineage": "abc123de-f456-7890-abcd-ef0123456789",
  "outputs": {
    "vpc_id": {
      "value": "vpc-0a1b2c3d4e5f67890",
      "type": "string",
      "sensitive": false
    },
    "db_password": {
      "value": "super-secret-password",
      "type": "string",
      "sensitive": true
    }
  },
  "resources": [
    {
      "module": "module.networking",
      "mode": "managed",
      "type": "aws_vpc",
      "name": "main",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 1,
          "attributes": {
            "id": "vpc-0a1b2c3d4e5f67890",
            "cidr_block": "10.0.0.0/16",
            "enable_dns_hostnames": true,
            "enable_dns_support": true,
            "tags": {
              "Environment": "prod",
              "ManagedBy": "terraform",
              "Name": "main-vpc"
            }
          },
          "dependencies": [
```

```
            "data.aws_availability_zones.available"
          ]
        }
      ]
    }
  ]
}
```

🔵 Key Fields Explained

```
version:           State file format version (currently 4)
terraform_version: Version of Terraform that last wrote this state
serial:            Monotonically increasing integer — increments on every
write
                   Used to detect concurrent modifications
lineage:           UUID generated when state is first created
                   Two state files with different lineage are different
state chains
                   Terraform refuses to apply if lineage mismatches

outputs:           All output values from the root module
                   Includes sensitive outputs (in PLAINTEXT — security
risk)

resources[]:       One entry per managed resource
  module:          Module path (empty string for root module)
  mode:            "managed" (resource) or "data" (data source)
  type:            Resource type (aws_vpc, aws_instance, etc.)
  name:            Resource name from config
  instances[]:     One per count index or for_each key
    attributes:    EVERY attribute — all stored here
    dependencies:  List of resource addresses this depends on
```

🔵 What State Does NOT Do

```
❌ State is NOT a backup of your infrastructure
   → It records what Terraform last knew, not the current cloud state
   → If something changes manually, state is stale until refresh

❌ State is NOT authoritative about what exists in the cloud
   → It's Terraform's cached view, not a real-time inventory

❌ State does NOT enforce access control
   → Anyone who can read the state file can see all attribute values
   → Including passwords, private keys, API keys stored in plaintext

✅ State IS the source of truth for Terraform's understanding
```

```
✅ State IS what enables idempotent plans (only show real changes)
✅ State IS what enables resource addressing and cross-references
```

🔵 The `serial` Field — Concurrency Protection

```
serial = 47 means this state has been written 47 times

When two applies try to write state simultaneously:
  Apply A reads serial=47, makes changes, tries to write serial=48
  Apply B reads serial=47, makes changes, tries to write serial=48

The second write FAILS — serial mismatch detected
This is why state locking (DynamoDB) is needed — prevents the race
```

🔵 Data Sources in State

```
# Data sources ARE stored in state — "mode": "data"
# They're refreshed on every plan
# Stored to enable cross-references and avoid re-querying every time

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
}

# In state:
# {
#   "mode": "data",
#   "type": "aws_ami",
#   "name": "ubuntu",
#   "instances": [{ "attributes": { "id": "ami-0abc123" ... } }]
# }
```

🔵 Short Interview Answer

> "State is Terraform's memory — a JSON file mapping every resource in config to its real-world cloud counterpart. It exists because Terraform needs to compute the difference between desired config and current reality without querying every resource in the cloud on every run. State stores resource IDs, all attribute values, dependency relationships, and output values. The `serial` field increments on every write and enables detecting concurrent modification attempts. Critically — state stores all attribute values in plaintext including sensitive values like passwords, which makes state file security as important as secrets management."

🔵 Gotchas & Edge Cases

- ⚠ **State is NOT encrypted by default** — local state and S3 state without SSE are plaintext JSON. Anyone with file access can read all passwords, keys, and tokens stored in resource attributes.
- ⚠ **Deleting state does NOT delete resources** — if you delete the state file, Terraform loses track of what it manages. The resources still exist in AWS but Terraform thinks they don't. Your next apply creates duplicates.
- `serial` **mismatch error** — if you manually edit state and the serial doesn't increment properly, or if you `state push` an old state, Terraform detects the mismatch and refuses.
- **State files grow over time** — old resource entries aren't automatically cleaned up. Large teams with many resources can have multi-MB state files.

🔵 Connections to Other Concepts

- → **Topic 36 (Remote Backends):** Where state is stored — local vs remote
- → **Topic 37 (Locking):** The serial field + DynamoDB prevents concurrent writes
- → **Topic 41 (Security):** State contains plaintext secrets — requires protection
- → **Category 11 Topic 89 (Drift):** State vs reality divergence

---

Topic 36: ⚠ Remote Backends — S3+DynamoDB, GCS, Terraform Cloud

---

🔵 What It Is (Simple Terms)

A **backend** defines where Terraform stores state. The default is a local file (`terraform.tfstate` in the working directory). Remote backends store state on a shared service — enabling team collaboration, state locking, versioning, and security controls.

> ⚠ Remote backends are non-negotiable in production. Every interviewer expects you to know S3+DynamoDB at minimum.

🔵 Why Remote Backends Are Essential

```
Local backend problems in teams:
  ❌ State file on one person's laptop — others can't plan/apply
  ❌ No state locking — concurrent applies corrupt state
  ❌ No versioning — can't recover from bad applies
  ❌ No access control — anyone with repo access can read secrets in state
  ❌ Disaster recovery — laptop dies = state lost

Remote backend solutions:
  ✅ Centralized state — everyone applies against same state
  ✅ State locking — prevents concurrent corruption
```

```
  ✅ Versioning — recover from bad applies (S3 versioning, TFC history)
  ✅ Access control — IAM/RBAC on who can read/write state
  ✅ Durability — S3/GCS with 11-nines durability
```

🔵 S3 + DynamoDB Backend — The AWS Standard

```
# versions.tf or backend.tf
terraform {
  backend "s3" {
    # ── S3 State Storage ──────────────────────────────────────────
    bucket  = "mycompany-terraform-state"
    key     = "prod/networking/terraform.tfstate"
    region  = "us-east-1"

    # ── Encryption ────────────────────────────────────────────────
    encrypt        = true
    kms_key_id     = "arn:aws:kms:us-east-1:123456789012:key/abc123"
    # Without kms_key_id: uses SSE-S3 (AES256)
    # With kms_key_id: uses SSE-KMS (customer-managed key)

    # ── DynamoDB Locking ──────────────────────────────────────────
    dynamodb_table = "terraform-state-locks"
    # Table must have primary key: LockID (String)

    # ── Access Control ────────────────────────────────────────────
    # Terraform uses the AWS provider's credentials
    # Typically an IAM role assumed by CI/CD
    # role_arn = "arn:aws:iam::123456789012:role/TerraformStateRole"
  }
}
```

S3 Bucket Setup

```
# bootstrap/main.tf — create the state bucket (run once manually)
resource "aws_s3_bucket" "terraform_state" {
  bucket = "mycompany-terraform-state"

  lifecycle { prevent_destroy = true }
}

resource "aws_s3_bucket_versioning" "state" {
  bucket = aws_s3_bucket.terraform_state.id
  versioning_configuration {
    status = "Enabled"   # CRITICAL — enables state recovery
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "state" {
```

```
    bucket = aws_s3_bucket.terraform_state.id
    rule {
      apply_server_side_encryption_by_default {
        sse_algorithm     = "aws:kms"
        kms_master_key_id = aws_kms_key.terraform_state.arn
      }
    }
  }

  resource "aws_s3_bucket_public_access_block" "state" {
    bucket                  = aws_s3_bucket.terraform_state.id
    block_public_acls       = true
    block_public_policy     = true
    ignore_public_acls      = true
    restrict_public_buckets = true
  }

  resource "aws_s3_bucket_policy" "state" {
    bucket = aws_s3_bucket.terraform_state.id
    policy = jsonencode({
      Version = "2012-10-17"
      Statement = [
        {
          Effect    = "Deny"
          Principal = "*"
          Action    = "s3:*"
          Resource  = [
            "${aws_s3_bucket.terraform_state.arn}",
            "${aws_s3_bucket.terraform_state.arn}/*"
          ]
          Condition = {
            Bool = { "aws:SecureTransport" = "false" }
          }
        }
      ]
    })
  }
```

DynamoDB Lock Table Setup

```
  resource "aws_dynamodb_table" "terraform_locks" {
    name         = "terraform-state-locks"
    billing_mode = "PAY_PER_REQUEST"   # no capacity planning needed
    hash_key     = "LockID"            # must be exactly "LockID"

    attribute {
      name = "LockID"
      type = "S"
    }
```

```
  lifecycle { prevent_destroy = true }
}
```

🔵 State Key Strategy — Organizing Multiple States

```
# Key structure: <environment>/<component>/terraform.tfstate
# Examples:
key = "prod/networking/terraform.tfstate"
key = "prod/security/terraform.tfstate"
key = "prod/data/terraform.tfstate"
key = "prod/compute/terraform.tfstate"
key = "staging/networking/terraform.tfstate"
key = "dev/networking/terraform.tfstate"

# Alternative: team-based structure
key = "platform/networking/terraform.tfstate"
key = "platform/security/terraform.tfstate"
key = "data-team/rds/terraform.tfstate"
key = "app-team/services/terraform.tfstate"
```

🔵 GCS Backend — The GCP Standard

```
terraform {
  backend "gcs" {
    bucket      = "mycompany-terraform-state"
    prefix      = "prod/networking"    # objects stored as
prod/networking/default.tfstate
    credentials = "/path/to/sa.json"  # or use ADC (recommended)
    # GCS provides built-in locking — no separate lock table needed
    # GCS provides built-in versioning — no configuration needed
  }
}
```

🔵 Azure Backend — Blob Storage

```
terraform {
  backend "azurerm" {
    resource_group_name  = "terraform-state-rg"
    storage_account_name = "mycompanytfstate"
    container_name       = "tfstate"
    key                  = "prod.networking.tfstate"
    # Azure Blob Storage provides built-in locking via lease mechanism
```

```
  }
}
```

🔵 Terraform Cloud Backend

```
terraform {
  cloud {
    organization = "mycompany"

    workspaces {
      name = "prod-networking"
      # OR use tags to select multiple workspaces:
      # tags = ["prod", "networking"]
    }
  }
}

# Terraform Cloud provides:
# ✅ State storage with full version history
# ✅ State locking built-in
# ✅ Encryption at rest and in transit
# ✅ Access control via Teams and permissions
# ✅ Remote plan/apply execution
# ✅ Sentinel policy enforcement
# ✅ VCS integration
# ✅ Cost estimation
```

🔵 Backend Configuration Best Practices

```
✅ Never hardcode credentials in backend config
   → Use IAM roles (EC2 instance profiles, ECS task roles, OIDC)
   → Backend config interpolation is NOT supported — use partial config

✅ Use partial backend config for DRY configuration
# backend.hcl — committed to repo (no secrets)
bucket         = "mycompany-terraform-state"
region         = "us-east-1"
dynamodb_table = "terraform-state-locks"
encrypt        = true

# backend.tfvars — committed (per-env key)
key = "prod/networking/terraform.tfstate"

# Initialize with partial config:
terraform init -backend-config=backend.hcl -backend-
config="key=prod/networking/terraform.tfstate"
```

```
✅ Enable S3 bucket versioning BEFORE storing state
✅ Enable MFA delete on state bucket for extra protection
✅ Separate state bucket per AWS account (not shared across accounts)
✅ Set lifecycle rules to expire old state versions after N days
```

🔵 Migrating from Local to Remote Backend

```
# Step 1: Add backend config to your .tf files
# Step 2: Run terraform init — it detects the backend change
terraform init

# Terraform prompts:
# "Do you want to copy existing state to the new backend?"
# Type "yes" — Terraform copies local state to S3

# Step 3: Verify remote state exists
terraform state list   # queries remote backend

# Step 4: Remove local state file
rm terraform.tfstate terraform.tfstate.backup
# (Git should never have tracked these — they should be in .gitignore)
```

🔵 Short Interview Answer

> "Remote backends store Terraform state on a shared service instead of a local file. The AWS standard is S3 for state storage plus DynamoDB for state locking. S3 should have versioning enabled (for recovery), server-side encryption with KMS (for security), public access blocked, and a bucket policy enforcing HTTPS. DynamoDB needs a table with `LockID` as the hash key — Terraform writes a lock record when apply starts and deletes it when done. GCS has built-in locking and versioning. Terraform Cloud bundles everything — state storage, locking, encryption, and CI/CD integration. In all cases, state encryption and access control are critical because state files contain plaintext credentials."

🔵 Common Interview Questions

Q: What happens if you forget to add the DynamoDB table to your S3 backend config?

> "You lose state locking. The backend still works — state is stored and retrieved from S3 — but nothing prevents two applies from running simultaneously. If two CI/CD pipelines trigger at the same time, both read the same state, both make different changes, and the second write overwrites the first. The losing apply's resource changes are 'orphaned' — they exist in AWS but are missing from state. Always configure the `dynamodb_table` argument and ensure the table exists before your first `terraform init`."

Q: How do you structure the S3 key for multiple environments and multiple state files?

> "The convention I use is `<environment>/<component>/terraform.tfstate` — for example `prod/networking/terraform.tfstate`, `prod/compute/terraform.tfstate`, `staging/networking/terraform.tfstate`. This groups by environment first so you can apply IAM policies at the environment prefix level — the prod CI role has access to `prod/*` keys, the staging role to `staging/*` keys. Some teams structure it by team ownership instead: `platform/networking`, `app-team/services`."

🔵 Gotchas & Edge Cases

- ⚠ **Backend config doesn't support variable interpolation** — `bucket = var.bucket_name` is invalid. Use partial backend config with `-backend-config` flags or hardcode the bucket name.
- ⚠ **Changing backend requires `terraform init -reconfigure`** — just editing the backend block isn't enough. You must re-run init to migrate.
- **S3 versioning must be enabled BEFORE first use** — you can't retroactively recover states that were written before versioning was enabled.
- **Cross-region state** — if your resources are in `eu-west-1` but state bucket is in `us-east-1`, that's fine. State bucket region should be your organization's primary region for compliance/latency reasons.

Topic 37: State Locking — Mechanism, Failure Scenarios

🔵 What It Is (Simple Terms)

State locking is a mechanism that **prevents two Terraform operations from modifying state simultaneously**. When an apply starts, it acquires a lock. Any other apply attempt during this time is blocked until the lock is released.

🔵 How Locking Works — S3 + DynamoDB

```
terraform apply starts:
  1. Acquires lock:
     → Writes item to DynamoDB table:
     {
       "LockID": "mycompany-tf-state/prod/networking/terraform.tfstate",
       "Info": {
         "ID": "f3b45c2d-1234-5678-abcd-ef0123456789",
         "Operation": "OperationTypeApply",
         "Who": "ci-runner@github-actions",
         "Version": "1.5.0",
         "Created": "2024-01-15T10:30:00.000Z",
         "Path": "mycompany-tf-state/prod/networking/terraform.tfstate"
       }
```

```
      }
      → Uses DynamoDB conditional write: only succeeds if item doesn't
exist
      → If item already exists: LOCK ACQUISITION FAILS

  2. Reads state from S3
  3. Creates plan
  4. Applies changes (creates/updates/destroys resources)
  5. Writes new state to S3 (serial + 1)
  6. Releases lock:
      → Deletes the DynamoDB item

Second concurrent apply:
  1. Tries to write lock item → DynamoDB conditional check fails
  2. Returns error with full lock info:
      "Error acquiring the state lock"
      Lock Info: ID=f3b45c2d... Who=ci-runner... Created=...
```

🔵 Which Operations Lock State

```
LOCKING operations (write + read):
  ✅ terraform apply
  ✅ terraform destroy
  ✅ terraform plan (acquires read lock in some backends)
  ✅ terraform state mv
  ✅ terraform state rm
  ✅ terraform state push
  ✅ terraform import
  ✅ terraform taint / untaint

NON-LOCKING operations (read-only):
  ✅ terraform state list
  ✅ terraform state show
  ✅ terraform state pull
  ✅ terraform output
```

🔵 Locking in Different Backends

```
S3 backend:
  → Requires separate DynamoDB table
  → No locking if dynamodb_table not configured
  → Lock granularity: one lock per state file key

GCS backend:
  → Built-in object locking
  → No additional setup required
  → Lock granularity: one lock per state object
```

```
Azure backend:
  → Built-in blob lease mechanism
  → No additional setup required

Terraform Cloud:
  → Built-in queue-based serialization
  → All runs for a workspace are queued
  → No concurrent runs possible — stronger than locking

Local backend:
  → .terraform.lock.info file in working directory
  → Works for single-user scenarios only
```

🔵 Lock Failure Scenarios

Scenario 1: Normal Lock Contention

```
# Engineer A is applying:
# Acquired lock at 10:00:00

# Engineer B tries to apply at 10:00:30:
Error: Error acquiring the state lock

Error message: ConditionalCheckFailedException: The conditional request
failed

Lock Info:
  ID:        f3b45c2d-1234-5678-abcd-ef0123456789
  Path:      mycompany-tf-state/prod/networking/terraform.tfstate
  Operation: OperationTypeApply
  Who:       alice@engineering-laptop
  Version:   1.5.0
  Created:   2024-01-15 10:00:00 UTC

# Resolution: Wait for Engineer A to finish
```

Scenario 2: Stuck Lock (Most Common Incident)

```
# CI/CD pipeline acquired lock, job was killed mid-apply
# Lock is stuck — will never be released automatically

# DynamoDB item: created 6 hours ago, no apply running

# Resolution: terraform force-unlock (see Topic 42)
```

Scenario 3: DynamoDB Table Missing

```
# Backend configured with dynamodb_table but table doesn't exist

Error: Error acquiring the state lock
  Error message: ResourceNotFoundException: Requested resource not found

# Resolution: Create the DynamoDB table
# This is a bootstrapping problem — chicken-and-egg
# Solution: Create the table manually or via a separate bootstrap config
```

Scenario 4: Insufficient Permissions

```
# CI/CD role has S3 access but not DynamoDB access

Error: Error acquiring the state lock
  Error message: AccessDeniedException: User is not authorized
  to perform: dynamodb:PutItem on resource: terraform-state-locks

# Resolution: Add DynamoDB permissions to the Terraform execution role
```

🔵 Required IAM Permissions for S3+DynamoDB Backend

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::mycompany-terraform-state",
        "arn:aws:s3:::mycompany-terraform-state/*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:DeleteItem",
        "dynamodb:DescribeTable"
      ],
```

```
      "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/terraform-state-locks"
    }
  ]
}
```

🔵 Short Interview Answer

> "State locking prevents concurrent applies from corrupting state. For S3 backend, Terraform writes a lock record to DynamoDB using a conditional write that only succeeds if no lock exists. If a lock is already held, the conditional write fails and the apply errors immediately with the full lock info — who holds it, when they acquired it. GCS and Azure have built-in locking. Terraform Cloud serializes all runs in a queue — even stronger than locking. The most common failure scenario is a stuck lock from a killed CI/CD job — the lock record stays in DynamoDB forever until manually removed with `terraform force-unlock`."

Topic 38: ⚠ `terraform state` Commands — `mv`, `rm`, `list`, `show`, `pull`, `push`

🔵 What It Is (Simple Terms)

`terraform state` is a subcommand group for **directly inspecting and manipulating the state file**. These are surgical tools — used for infrastructure refactoring, incident recovery, and state debugging. Used wrong, they cause state corruption.

> ⚠ State commands are heavily tested in senior interviews. Know each command's purpose, syntax, and blast radius.

🔵 `terraform state list` — Inspect What's in State

```
# List all resources in state
terraform state list

# Example output:
data.aws_availability_zones.available
data.aws_caller_identity.current
module.networking.aws_subnet.private[0]
module.networking.aws_subnet.private[1]
module.networking.aws_subnet.public[0]
module.networking.aws_vpc.main
aws_iam_role.app
aws_instance.web["api"]
aws_instance.web["worker"]
```

```
aws_s3_bucket.data

# Filter by pattern
terraform state list 'module.networking.*'
terraform state list 'aws_instance.*'
terraform state list '*security_group*'
```

🔵 `terraform state show` — Inspect a Specific Resource

```
# Show all attributes of a specific resource
terraform state show aws_instance.web

# Example output:
# aws_instance.web:
resource "aws_instance" "web" {
    ami                          = "ami-0c55b159cbfafe1f0"
    arn                          = "arn:aws:ec2:us-east-
1:123456789012:instance/i-0abc123def456"
    id                           = "i-0abc123def456"
    instance_type                = "t3.medium"
    private_ip                   = "10.0.1.45"
    public_ip                    = "54.12.34.56"
    subnet_id                    = "subnet-0123456789abcdef"
    vpc_security_group_ids       = ["sg-0123456789abcdef"]
    tags                         = {
        "Environment" = "prod"
        "Name"        = "prod-web"
    }
    # ... all attributes ...
}

# Show a module resource
terraform state show 'module.networking.aws_vpc.main'

# Show a count-indexed resource
terraform state show 'aws_instance.workers[0]'

# Show a for_each resource (note the quotes)
terraform state show 'aws_instance.web["api"]'
```

🔵 `terraform state mv` — Move/Rename Resources in State

```
# Move a resource (rename without destroying)
terraform state mv aws_instance.web aws_instance.web_server

# Move to/from a module
terraform state mv aws_vpc.main module.networking.aws_vpc.main
```

```
# Cross-state file move (with -state and -state-out)
terraform state mv \
  -state=source.tfstate \
  -state-out=destination.tfstate \
  aws_instance.web \
  aws_instance.web

# Rename count-indexed resource
terraform state mv 'aws_instance.web[0]' 'aws_instance.web["server-a"]'

# Dry run — see what would happen without making changes
terraform state mv -dry-run aws_instance.web aws_instance.web_server
```

When to use `state mv`:

```
✅ Renaming a resource in config without destroying it
   (preferred: use moved block in Terraform 1.1+)
✅ Moving a resource into a module
✅ Cross-state file migration
✅ Migrating from count to for_each (see Category 11 Topic 98)
✅ Renaming a module without affecting resources inside it
```

🔵 `terraform state rm` — Remove Resource from State

```
# Remove a resource from state (does NOT destroy the actual resource)
terraform state rm aws_instance.web

# Remove a module and all resources within it
terraform state rm module.networking

# Remove a count-indexed resource
terraform state rm 'aws_instance.workers[2]'

# Remove a for_each resource
terraform state rm 'aws_instance.web["api"]'

# Remove multiple resources
terraform state rm aws_instance.web aws_iam_role.app
```

When to use `state rm`:

```
✅ Removing a resource from Terraform management without destroying it
   (you want to manage it manually or with another tool)
✅ Recovery when state has a phantom resource that doesn't exist
   (state says it exists, AWS says it doesn't — remove from state)
```

```
✅ Removing a resource before importing it under a new address
✅ Cleaning up data sources that are failing refresh
```

What happens after `state rm`:

```
# After removing aws_instance.web from state:
terraform plan
# Shows: + aws_instance.web will be created
# (Terraform thinks it doesn't exist — will try to recreate)
# This is usually NOT what you want unless you've also removed from config
```

🔵 `terraform state pull` — Download State Locally

```
# Pull state from remote backend to stdout
terraform state pull

# Save to a local file
terraform state pull > backup-$(date +%Y%m%d-%H%M%S).tfstate

# Pretty-print and inspect
terraform state pull | jq '.'

# Check serial number
terraform state pull | jq '.serial'

# List all resource IDs
terraform state pull | jq '.resources[].instances[].attributes.id'

# Find a specific resource
terraform state pull | jq '.resources[] | select(.type ==
"aws_db_instance")'
```

Use cases:

```
✅ Backup before risky operations
✅ Offline inspection and analysis
✅ Cross-state migration (pull → modify → push)
✅ Debugging — see exactly what's in state
✅ Audit — who created what resource with what attributes
```

🔵 `terraform state push` — Upload State

```
# Push a local state file to the remote backend
# ⚠ DANGEROUS — overwrites remote state
terraform state push backup.tfstate

# Force push even if serial is lower than remote
# ⚠ VERY DANGEROUS — bypasses serial protection
terraform state push -force backup.tfstate
```

When to use `state push`:

```
✅ Restoring from backup after corruption
✅ Completing a cross-state migration
✅ Recovering from a failed migration attempt

❌ NEVER use -force unless you have no other option
❌ NEVER push in a team environment without locking coordination
```

🔵 State Command Safety Rules

```
ALWAYS do before any state operation:
  1. terraform state pull > backup-$(date +%Y%m%d).tfstate
  2. Verify the backup is readable: cat backup-*.tfstate | jq '.serial'
  3. Coordinate with team — notify in Slack/chat

AFTER every state operation:
  1. terraform state list — verify expected resources are present
  2. terraform plan — verify 0 unexpected changes
  3. If unexpected changes: restore from backup immediately

NEVER:
  ❌ state push -force without verified backup
  ❌ state mv without running plan after
  ❌ state rm on a running production resource (destroys orphan risk)
  ❌ Direct JSON editing of state files
```

🔵 Short Interview Answer

> "`terraform state` commands are surgical tools for inspecting and manipulating state. `list` shows all managed resources, `show` displays all attributes of a specific resource. `mv` renames or moves resources in state without touching real infrastructure — the preferred approach is now `moved` blocks for same-state renames. `rm` removes a resource from Terraform management without destroying it — used for recovering phantom resources or unmanaging resources. `pull` downloads state locally — essential for backups before risky operations. `push` uploads state — used for disaster

> recovery, dangerous with `-force`. The golden rule: always `pull` a backup before any state manipulation, and always run `plan` after to verify no unexpected changes."

Topic 39: ⚠ Splitting Large State — When and How

🔵 What It Is (Simple Terms)

Large Terraform configurations that manage hundreds of resources in a single state file become slow, risky, and hard to collaborate on. Splitting means dividing one large config into multiple smaller configs, each with its own state file.

🔵 Warning Signs You Need to Split

```
⚠ terraform plan takes > 5 minutes
⚠ State file is > 5MB
⚠ > 200 resources in one state file
⚠ Multiple teams need to apply to the same config
⚠ A single bug affects all infrastructure
⚠ Fear of making changes due to blast radius
⚠ Long apply times block urgent fixes
⚠ Unrelated resources in the same risk domain
```

🔵 Splitting Principles

```
Split by: DOMAIN (what the resources do)
  networking/    ← VPC, subnets, routes, peering
  security/      ← IAM roles, security groups, KMS keys
  data/          ← RDS, ElastiCache, S3 data buckets
  compute/       ← EKS, EC2, ASG, Lambda
  platform/      ← ECS cluster, ALB, DNS
  applications/  ← Per-service resources

Split by: TEAM OWNERSHIP
  platform-team/
  data-team/
  app-team/

Split by: RATE OF CHANGE
  stable/        ← VPC, IAM — changes rarely, high risk to change
  dynamic/       ← ASG, ECS tasks — changes frequently, lower risk

DO NOT split by:
  ❌ Random convenience — splitting creates coordination overhead
```

```
    Too granular (1-5 resources per state) — unnecessary overhead
  ❌ Circular dependencies — A needs B and B needs A
```

🔵 Cross-Stack Reference Patterns

```
# When state A is split from state B, you need a way to
# reference outputs from A in B's config

# ── Pattern 1: terraform_remote_state (tight coupling) ────────────────
data "terraform_remote_state" "networking" {
  backend = "s3"
  config  = {
    bucket = "mycompany-terraform-state"
    key    = "prod/networking/terraform.tfstate"
    region = "us-east-1"
  }
}

resource "aws_instance" "app" {
  # Reference networking stack outputs
  subnet_id              =
data.terraform_remote_state.networking.outputs.private_subnet_ids[0]
  vpc_security_group_ids =
[data.terraform_remote_state.networking.outputs.app_security_group_id]
}

# ── Pattern 2: SSM Parameter Store (loose coupling — preferred) ────────
# networking stack writes outputs to SSM:
resource "aws_ssm_parameter" "vpc_id" {
  name  = "/prod/networking/vpc_id"
  type  = "String"
  value = aws_vpc.main.id
}

resource "aws_ssm_parameter" "private_subnet_ids" {
  name  = "/prod/networking/private_subnet_ids"
  type  = "StringList"
  value = join(",", aws_subnet.private[*].id)
}

# application stack reads from SSM:
data "aws_ssm_parameter" "vpc_id" {
  name = "/prod/networking/vpc_id"
}

data "aws_ssm_parameter" "private_subnet_ids" {
  name = "/prod/networking/private_subnet_ids"
}

resource "aws_instance" "app" {
```

```
  subnet_id = split(",", data.aws_ssm_parameter.private_subnet_ids.value)
[0]
}
```

🔵 The Migration Process (Zero Downtime)

```
# Step 1: Back up current state
terraform state pull > monolith-backup-$(date +%Y%m%d).tfstate

# Step 2: Create new directory structure
mkdir -p stacks/{networking,security,compute}

# Step 3: Create new configs (copy resource blocks from monolith)

# Step 4: Initialize new stacks (empty state)
cd stacks/networking && terraform init

# Step 5: Move state entries (for each resource to migrate)
terraform state mv \
  -state=../../monolith-backup.tfstate \
  -state-out=stacks/networking/networking.tfstate \
  aws_vpc.main \
  aws_vpc.main

# Repeat for all networking resources

# Step 6: Push new state to remote
cd stacks/networking
terraform state push networking.tfstate

# Step 7: Run plan on new stack — must show 0 changes
terraform plan  # should show: No changes.

# Step 8: Remove resources from monolith config
# (don't remove from monolith state yet — verify first)

# Step 9: Run plan on monolith — verify only moved resources show
# as "will be destroyed" (from monolith perspective)

# Step 10: Remove from monolith state
terraform state rm aws_vpc.main
# ... remove all migrated resources

# Step 11: Final verification
cd stacks/networking && terraform plan  # 0 changes
cd monolith && terraform plan           # 0 changes on remaining resources
```

🔵 Dependency Ordering After Split

```
Before split: single config, all resources in one graph
After split:  multiple configs, must apply in dependency order

Correct apply order:
  1. networking    (VPC, subnets — no dependencies)
  2. security      (IAM, SGs — may depend on VPC ID)
  3. data          (RDS — depends on subnets, SGs)
  4. compute       (EKS, EC2 — depends on subnets, SGs, RDS endpoint)
  5. applications  (services — depends on EKS cluster, RDS)

In CI/CD: orchestrate as pipeline stages
  Stage 1: networking → security (parallel)
  Stage 2: data (depends on 1)
  Stage 3: compute (depends on 1+2)
  Stage 4: applications (depends on 3)
```

🔵 Short Interview Answer

> "Split large state when plans take more than 5 minutes, when multiple teams need independent apply cycles, or when a single failure can affect all infrastructure. Split by domain — networking, security, data, compute, applications — following team ownership boundaries and dependency direction. The migration uses `terraform state mv` with `-state` and `-state-out` flags to move state entries between files, then updating configs to remove from source and add to destination. After migration, plans on both stacks should show zero changes. Cross-stack references switch from direct attribute access to either `terraform_remote_state` (tight coupling) or SSM Parameter Store (loose coupling — preferred)."

Topic 40: Drift Detection — `plan`, `refresh`, Real-World Handling

🔵 What It Is (Simple Terms)

Drift is when the real infrastructure diverges from what Terraform's state believes. Drift detection is the process of identifying and reconciling these differences.

🔵 How Terraform Detects Drift

```
terraform plan (default behavior):
  1. Read state → get list of managed resources + last-known attributes
  2. REFRESH: call provider APIs for each resource to get current
attributes
  3. Compare: current reality vs config
```

```
  4. Show diff: what would change to bring reality to config

This refresh step IS drift detection for managed resources
```

🔵 The Three Commands for Drift

```
# 1. terraform plan (detect + show what config wants)
terraform plan
# Shows drift AND the changes needed to fix it
# Combines: "here's what changed" + "here's what I'll do about it"

# 2. terraform apply -refresh-only (detect only, accept reality)
terraform plan -refresh-only    # Preview what drifted
terraform apply -refresh-only   # Update state to match reality
# Does NOT revert changes — accepts manual changes into state

# 3. terraform refresh (deprecated but still works)
terraform refresh
# Updates state file to match reality
# Equivalent to terraform apply -refresh-only
# Deprecated in TF 0.15 — prefer apply -refresh-only

# 4. Disable refresh (fast but blind)
terraform plan -refresh=false
# Skips provider API calls — fastest but won't detect drift
# Use only when you're certain no manual changes occurred
```

🔵 Types of Drift and How to Handle Each

```
Type 1: ATTRIBUTE drift
  What happened: Someone changed instance_type from t3.medium to t3.large
in console
  terraform plan shows: ~ instance_type = "t3.large" -> "t3.medium"

  Decision:
    Was the change intentional?
      YES → Update config to t3.large, commit, plan should be clean
      NO  → terraform apply to revert to t3.medium

Type 2: EXISTENCE drift (resource deleted externally)
  What happened: Security group was manually deleted
  terraform plan shows: + aws_security_group.web will be created

  Decision:
    Should it be recreated?
      YES → terraform apply (recreates it)
      NO  → terraform state rm + remove from config
```

```
Type 3: INVISIBLE drift (unmanaged resource created externally)
  What happened: Someone manually created an S3 bucket
  terraform plan shows: (nothing — Terraform doesn't know about it)

  Detection: Requires external tools (AWS Config, Driftctl, CloudTrail)
  Decision:
    Should Terraform manage it?
      YES → terraform import to bring it under management
      NO  → leave as-is (document why it's unmanaged)
```

🔵 `refresh-only` — The Right Tool for Intentional Changes

```
# Scenario: Ops team manually scaled up an ASG during an incident
# You want to accept this change, not revert it

# Step 1: See what drifted
terraform plan -refresh-only
# Shows: ~ desired_capacity = 3 -> 5  (AWS has 5, state has 3)

# Step 2: Accept the drift into state
terraform apply -refresh-only
# Updates state: desired_capacity is now recorded as 5
# Your config still says 3

# Step 3: Codify the change (optional but recommended)
# Update variable or config to desired_capacity = 5
# Commit and push

# Step 4: Verify
terraform plan
# Now shows: ~ desired_capacity = 5 -> 3 (config wants 3, reality is 5)
# Your choice: update config to 5 or revert to 3
```

🔵 Drift in CI/CD — Automated Detection

```
# GitHub Actions workflow: scheduled drift detection
name: Drift Detection
on:
  schedule:
    - cron: '0 8 * * 1-5'   # 8am every weekday

jobs:
  detect-drift:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
```

```yaml
      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v2

      - name: Terraform Plan (drift check)
        run: |
          terraform init
          terraform plan -detailed-exitcode
          # Exit codes:
          # 0 = no changes (no drift)
          # 1 = error
          # 2 = changes detected (drift!)
        continue-on-error: true

      - name: Alert on drift
        if: steps.plan.outputs.exitcode == '2'
        run: |
          curl -X POST $SLACK_WEBHOOK \
            -d '{"text":"⚠ Terraform drift detected in prod/networking!"}'
```

🔵 Short Interview Answer

> "Drift detection works through Terraform's refresh mechanism — during `terraform plan`, it calls provider APIs for every managed resource to get current attribute values, then compares them to state. If they differ, the plan shows what changed. `terraform apply -refresh-only` accepts manual changes into state without reverting them — useful after emergency incident response where ops manually scaled resources. Drift from deleted resources shows as 'will be created'. Invisible drift (manually created resources Terraform doesn't know about) requires external tools like AWS Config or Driftctl. In CI/CD, scheduled plans with `-detailed-exitcode` flag (exit code 2 = changes detected) enable automated drift alerting."

Topic 41: State File Security — Encryption, Access Control, Secret Exposure Risk

---

🔵 What It Is (Simple Terms)

State files contain sensitive data — every attribute of every resource, including passwords, private keys, API tokens, and connection strings — stored in **plaintext JSON**. State security means protecting this data from unauthorized access.

🔵 What's Exposed in State

```
// Real example of what's in state — ALL plaintext:
{
```

```
  "resources": [
    {
      "type": "aws_db_instance",
      "name": "production",
      "instances": [{
        "attributes": {
          "id": "prod-database",
          "endpoint": "prod-database.abc123.us-east-
1.rds.amazonaws.com:5432",
          "password": "super-secret-prod-password-123",   // ← PLAINTEXT
          "username": "admin",
          "connection_url": "postgres://admin:super-secret-prod-password-
123@..."
        }
      }]
    },
    {
      "type": "tls_private_key",
      "name": "app",
      "instances": [{
        "attributes": {
          "private_key_pem": "-----BEGIN RSA PRIVATE KEY-----\n...",  // ←
PLAINTEXT
          "public_key_pem": "-----BEGIN PUBLIC KEY-----\n..."
        }
      }]
    }
  ],
  "outputs": {
    "db_password": {
      "value": "super-secret-prod-password-123",   // ← PLAINTEXT (even
sensitive=true outputs!)
      "sensitive": true
    }
  }
}
```

> ⚠ `sensitive = true` on a variable or output ONLY redacts from CLI display. The value is ALWAYS stored in plaintext in state.

🔵 Encryption at Rest

```
# S3 backend with KMS encryption
terraform {
  backend "s3" {
    bucket     = "mycompany-terraform-state"
    key        = "prod/terraform.tfstate"
    region     = "us-east-1"
    encrypt    = true                    # enables server-side encryption
    kms_key_id = "alias/terraform-state" # use customer-managed KMS key
```

```
    # Without kms_key_id: uses SSE-S3 (AWS-managed key)
    # With kms_key_id: uses SSE-KMS (you control the key)
  }
}

# SSE-S3 vs SSE-KMS:
# SSE-S3: AWS manages the key, automatic, free
#         Anyone with S3 access can read — encryption doesn't help against
S3 permissions
# SSE-KMS: You manage the key, costs $1/month + API calls
#          Separate KMS key policy controls decryption
#          Even if S3 bucket access is granted, KMS key policy can deny
decryption
#          All KMS API calls are logged in CloudTrail — audit trail
#          Preferred for production secrets
```

🔵 Access Control

```
// Least-privilege IAM policy for Terraform execution role
{
  "Statement": [
    {
      // Read/write state
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::mycompany-terraform-state/prod/*"
    },
    {
      // List bucket (needed for terraform init)
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::mycompany-terraform-state",
      "Condition": {
        "StringLike": { "s3:prefix": ["prod/*"] }
      }
    },
    {
      // State locking
      "Effect": "Allow",
      "Action": ["dynamodb:GetItem", "dynamodb:PutItem",
"dynamodb:DeleteItem"],
      "Resource": "arn:aws:dynamodb:us-east-
1:123456789012:table/terraform-state-locks"
    },
    {
      // Decrypt state with KMS
      "Effect": "Allow",
      "Action": ["kms:GenerateDataKey", "kms:Decrypt"],
      "Resource": "arn:aws:kms:us-east-1:123456789012:key/abc123"
    }
```

```
  ]
}
```

🔵 Access Control Strategy

```
Separate IAM roles per environment:
  TerraformRole-Dev    → access to dev/* state keys
  TerraformRole-Prod   → access to prod/* state keys
  TerraformRole-Read   → GetObject only (for drift detection jobs, no
writes)

S3 bucket policies:
  Deny HTTP (only HTTPS)
  Deny public access (all 4 block public access settings)
  Require encryption on PutObject
  Enable MFA Delete for extra protection

CloudTrail:
  Log all S3 data events on state bucket
  Log all KMS API calls
  → Full audit trail: who accessed state, when, from where
```

🔵 Best Practice: Minimize Secrets in State

```
# ❌ Approach that leaks secrets into state
resource "aws_db_instance" "main" {
  password = var.db_password    # stored in state
}

# ✅ Better approach: generate and store in Secrets Manager
resource "random_password" "db" {
  length  = 32
  special = true
}

resource "aws_secretsmanager_secret_version" "db_password" {
  secret_id     = aws_secretsmanager_secret.db.id
  secret_string = random_password.db.result
  # random_password.db.result IS still in state
  # BUT it's also in Secrets Manager for application use
}

resource "aws_db_instance" "main" {
  manage_master_user_password = true   # AWS manages rotation, not
Terraform
  # ← Password never appears in state at all!
}
```

```
# Even better for RDS: use manage_master_user_password = true
# This tells AWS to generate and manage the password via Secrets Manager
# Terraform never sees the password at all
```

🔵 Terraform Cloud State Security

```
Terraform Cloud provides:
  ✅ Encryption at rest (AES-256)
  ✅ Encryption in transit (TLS)
  ✅ Access control via Organizations, Teams, Workspace permissions
  ✅ State versions with full audit log
  ✅ Granular permissions: read state, write state, queue plans
  ✅ No need to manage your own S3 bucket and KMS key
  ✅ Sentinel policies can enforce state encryption requirements
```

🔵 Short Interview Answer

> "State files are a significant security risk because they contain every resource attribute in plaintext — including passwords, private keys, API tokens, and connection strings. This is true even for values marked `sensitive = true`, which only redacts CLI output. Protection requires three layers: encryption at rest (S3 SSE-KMS with customer-managed keys for independent access control), strict IAM access control with separate roles per environment and least-privilege permissions, and audit logging via CloudTrail on the state bucket and KMS key. The best mitigation is minimizing secrets in state entirely — use RDS's `manage_master_user_password` flag, fetch secrets from Vault at runtime, or use `aws_secretsmanager_secret_version` to keep secrets in Secrets Manager where rotation and access control are purpose-built."

Topic 42: `terraform force-unlock` — When and How to Use Safely

🔵 What It Is (Simple Terms)

`terraform force-unlock` manually removes a state lock when the process that held it has died and the lock is stuck. It's an emergency tool — used when a killed CI/CD job, crashed terminal, or network failure leaves a stale lock preventing all subsequent applies.

🔵 When a Lock Gets Stuck

```
Normal lifecycle:
  terraform apply → acquires lock → applies changes → releases lock → done

Stuck lock scenarios:
  1. CI/CD job was killed (timeout, manual cancel, OOM)
  2. terraform process was killed with SIGKILL (kill -9)
  3. Network failure mid-apply severed connection to DynamoDB
  4. Machine powered off or crashed during apply
  5. Terraform panic/crash before lock release

Result: DynamoDB has a lock record with no active owner
        Every subsequent apply fails with "state is locked"
```

🔵 Identifying a Stuck Lock

```
# Error from any terraform operation:
Error: Error acquiring the state lock

  Error message: ConditionalCheckFailedException
  Lock Info:
    ID:        f3b45c2d-1234-5678-abcd-ef0123456789
    Path:      mycompany-tf-state/prod/networking/terraform.tfstate
    Operation: OperationTypeApply
    Who:       ci-runner@github-actions-abc123
    Version:   1.5.0
    Created:   2024-01-15 02:30:00 UTC    ← 6 hours ago!

# Key indicators it's stuck (not just concurrent):
# 1. Created time is in the past (minutes or hours ago)
# 2. No active CI/CD jobs exist for this workspace
# 3. The "Who" references a job or session that no longer exists
```

🔵 Pre-Unlock Verification Checklist

```
BEFORE running force-unlock, answer ALL of these:

1. Is there an active apply running?
   → Check GitHub Actions / GitLab CI / Jenkins for running jobs
   → Check your team's chat for anyone actively applying
   → Check AWS CloudTrail for recent API calls from the Terraform role

2. How old is the lock?
   → Minutes old: could be a slow apply, wait longer
   → Hours old: almost certainly stuck, safe to unlock

3. Is the state in a consistent state?
```

```
    → Was the apply making progress when it was killed?
    → Pull state and check serial: terraform state pull | jq '.serial'
    → If serial seems wrong, investigate before unlocking

4. Who will be affected?
    → Notify team before force-unlocking
    → Ensure no one else will apply immediately after you unlock
```

🔵 Running `force-unlock`

```
# Get the lock ID from the error message
# Lock ID: f3b45c2d-1234-5678-abcd-ef0123456789

# Force unlock
terraform force-unlock f3b45c2d-1234-5678-abcd-ef0123456789

# Output:
# Do you really want to force-unlock?
# Terraform will remove the lock on the remote state.
# This will allow local Terraform commands to modify this state, even
though it
# may still be in use. Only 'yes' will be accepted to confirm.
#
# Enter a value: yes
#
# Terraform state has been successfully unlocked!

# Skip confirmation prompt (use in automation carefully)
terraform force-unlock -force f3b45c2d-1234-5678-abcd-ef0123456789
```

🔵 Direct DynamoDB Recovery (Emergency)

```
# If terraform force-unlock itself fails (Terraform can't connect to
backend):

# Find the lock record
aws dynamodb get-item \
  --table-name terraform-state-locks \
  --key '{"LockID": {"S": "mycompany-tf-
state/prod/networking/terraform.tfstate"}}' \
  --region us-east-1

# Delete the lock record directly
aws dynamodb delete-item \
  --table-name terraform-state-locks \
  --key '{"LockID": {"S": "mycompany-tf-
state/prod/networking/terraform.tfstate"}}' \
```

```
  --region us-east-1

# This is the nuclear option — only when terraform force-unlock fails
```

🔵 Post-Unlock Steps

```
# After unlocking, ALWAYS do these before allowing new applies:

# Step 1: Pull current state and verify it's intact
terraform state pull | jq '.serial, (.resources | length)'
# Check: serial looks right, resource count is expected

# Step 2: Run a plan to assess state consistency
terraform plan
# Look for: unexpected creates, destroys, or updates
# If the killed apply was mid-way, some resources may be in partial state

# Step 3: Apply if plan looks correct
terraform apply

# Step 4: If plan shows unexpected changes:
# Investigate whether the killed apply left resources in inconsistent
state
# May need manual reconciliation of partially-created resources
```

🔵 Short Interview Answer

> "`terraform force-unlock` removes a stuck state lock record from DynamoDB when the apply that held it has crashed or been killed. The critical rule: never force-unlock without verifying the owning process is genuinely dead. Check CI/CD dashboards, check the lock's creation time — a lock from 6 hours ago for a job that no longer exists is safe to unlock. A 30-second-old lock might be a currently-running slow apply. After unlocking, always run `terraform state pull` to verify state integrity and `terraform plan` before any new apply. If `force-unlock` itself fails due to connectivity issues, you can directly delete the lock item from DynamoDB using the AWS CLI."

Topic 43: Workspaces — What They Are, OSS vs TFC

🔵 What It Is (Simple Terms)

Workspaces are a way to have **multiple state files from a single Terraform configuration directory**. Each workspace gets its own isolated state — allowing the same code to manage multiple environments without duplicating configuration.

🔵 How Workspaces Work

```
# Default workspace (always exists)
terraform workspace list
# * default         ← current workspace marked with *

# Create a new workspace
terraform workspace new staging
# Created and switched to workspace "staging"!
# You're now on a new, empty workspace.

terraform workspace list
#   default
# * staging         ← now on staging

# Create and switch more
terraform workspace new prod
terraform workspace list
#   default
#   staging
# * prod

# Switch between workspaces
terraform workspace select staging
terraform workspace select default

# Show current workspace
terraform workspace show
# prod

# Delete a workspace (must not be current)
terraform workspace select default
terraform workspace delete staging
```

🔵 Where State Is Stored Per Workspace

```
Local backend:
  terraform.tfstate              ← default workspace
  terraform.tfstate.d/
    staging/
      terraform.tfstate          ← staging workspace
    prod/
      terraform.tfstate          ← prod workspace

S3 backend:
  Default workspace key:
    mycompany-tf-state/prod/networking/terraform.tfstate

  Non-default workspace key:
```

```
    mycompany-tf-state/env:/staging/prod/networking/terraform.tfstate
    mycompany-tf-state/env:/prod/prod/networking/terraform.tfstate
    ← Note: "env:/<workspace_name>/" prefix is prepended automatically
```

🔵 OSS Workspaces vs Terraform Cloud Workspaces

```
┌──────────────────────────────────────────────────────────────────┐
│           OSS Workspaces vs Terraform Cloud Workspaces           │
│                                                                  │
│  OSS (Open Source) Workspaces:                                   │
│  ✅ Multiple state files from one config directory               │
│  ✅ Free, built into terraform CLI                               │
│  ✅ Simple for single-user or small team use                     │
│  ❌ Same code for all workspaces (can't have workspace-specific TF) │
│  ❌ No per-workspace variables/secrets management                │
│  ❌ No per-workspace access control                              │
│  ❌ No remote execution, no UI                                   │
│  ❌ S3 key path changes with workspace name (surprising behavior) │
│  Used for: lightweight environment separation, dev/test          │
│                                                                  │
│  Terraform Cloud Workspaces:                                     │
│  ✅ Fully isolated: separate state, variables, permissions per WS │
│  ✅ Per-workspace Terraform variables AND environment variables  │
│  ✅ Per-workspace RBAC — different teams for different workspaces │
│  ✅ Remote execution — runs happen in TFC infrastructure         │
│  ✅ VCS integration — connect a workspace to a branch            │
│  ✅ Run history, approval gates, Sentinel policies               │
│  ✅ Cost estimation per workspace                                │
│  ✅ Better suited to full production workflows                   │
│  Different concept: TFC workspaces = separate working directories │
└──────────────────────────────────────────────────────────────────┘
```

🔵 Short Interview Answer

> "OSS Terraform workspaces allow multiple isolated state files from a single configuration directory — each workspace gets its own state. You create them with `terraform workspace new`, switch with `terraform workspace select`, and the current workspace name is available as `terraform.workspace` in code. They're useful for simple environment separation. Terraform Cloud workspaces are a different concept — they're closer to separate working directories, each with their own state, variables, secrets, access control, and remote execution. TFC workspaces are production-grade; OSS workspaces are lightweight tools with significant limitations."

Topic 44: ⚠️ Workspaces vs Separate State Files — When to Use Which

---

🔵 What It Is (Simple Terms)

This is one of the most common architectural decisions in Terraform — should you use workspaces to manage multiple environments, or separate directories with separate state files? The answer matters significantly in production.

🔵 The Core Architectural Difference

```
Workspaces approach:
  One directory, one set of .tf files
  Multiple workspaces = multiple state files
  Code is IDENTICAL across all environments
  Variables differ via workspace-specific tfvars

  prod/             ← single directory
    main.tf
    variables.tf
    terraform.tfvars
    prod.tfvars
    staging.tfvars

  terraform workspace select prod
  terraform apply -var-file=prod.tfvars

Separate directories approach:
  Multiple directories, potentially shared modules
  Each directory has its own state

  environments/
    prod/
      main.tf       ← can differ from staging
      backend.tf
      terraform.tfvars
    staging/
      main.tf       ← can have different resources
      backend.tf
      terraform.tfvars
  modules/          ← shared logic
    vpc/
    rds/
```

🔵 When Workspaces Work Well

```
✅ Environments are genuinely identical in structure
   (same resources, same modules, only values differ)
```

```
✅ Simple configurations without complex per-env logic
✅ Ephemeral environments: create and destroy for testing
   (feature branch environments, PR environments)
✅ Small teams without complex access control needs
✅ The same person/role manages all workspaces

Example: Network testing
  workspace "test-feature-123": spin up, test, destroy
  workspace "test-feature-456": spin up, test, destroy
  → Temporary, isolated, same config, safe to share
```

🔵 When Separate State Files Are Required

```
❌ Environments need different resources
   (prod has RDS multi-AZ, dev has a single SQLite — completely different)

❌ Different teams manage different environments
   (platform team manages prod, developers manage dev)
   Workspaces don't support per-workspace IAM/RBAC in OSS

❌ Different apply approval processes
   (prod requires two approvals, dev is self-service)

❌ Risk isolation is required
   (a bug in staging Terraform config should NEVER affect prod)
   With workspaces: one wrong `workspace select` = prod apply from staging
config

❌ Environments have significant structural differences
   (prod has WAF, monitoring, backup — dev has none)

❌ Compliance requires environment isolation
   (SOC2, PCI-DSS often require strict environment separation)
```

🔵 The `workspace select` Mistake — Why Workspaces Are Dangerous

```
# The most dangerous mistake with workspaces:

# Engineer intends to apply to staging:
terraform workspace select staging   # MEANT to run this
terraform apply -var-file=prod.tfvars  # accidentally used prod vars

# OR:
terraform workspace select prod      # ACCIDENTALLY ran this
terraform apply -var-file=staging.tfvars  # now applying staging config to
prod state!
```

```
# There is NOTHING stopping you from running prod config on staging
workspace
# or staging config on prod workspace
# The workspace name is just a state file selector — it doesn't enforce
which
# tfvars file you use

# With separate directories:
cd environments/prod     # ← you have to physically be in the right
directory
terraform apply          # ← there's no accidental cross-environment apply
                         #   (still possible with wrong -var-file but much
harder)
```

🔵 Decision Framework

```
Q: Are all environments structurally identical (same resources, different
values)?
  NO → Use separate directories

Q: Do different teams need to manage different environments?
  YES → Use separate directories

Q: Are these ephemeral/short-lived environments?
  YES → Workspaces are fine

Q: Does prod need more resources than dev (multi-AZ, WAF, monitoring)?
  YES → Use separate directories

Q: Is compliance/audit separation required?
  YES → Use separate directories

Q: Is this a small team where one person manages all envs?
  YES → Workspaces might be acceptable

Rule of thumb:
  Production workloads → Separate directories with separate state files
  Ephemeral test environments → Workspaces are great
```

🔵 Short Interview Answer

> "Workspaces give you multiple state files from one config directory. They work well for ephemeral environments — feature branches, PR environments — where the config is genuinely identical and you want to spin up and destroy quickly. For persistent environments like dev/staging/prod, separate directories are strongly preferred. The main problem with workspaces for production environments: there's no enforcement that you're using the right workspace with the right tfvars — you can accidentally apply staging config to the prod workspace state. Separate directories provide physical

> separation that prevents cross-environment accidents. They also support different resource configurations per environment and per-environment access control."

Topic 45: ➕ `terraform.workspace` Interpolation in Code

🔵 What It Is (Simple Terms)

`terraform.workspace` is a built-in value that returns the **name of the currently selected workspace**. It can be used anywhere in your configuration to make resources workspace-aware — naming resources differently, selecting different configurations, or using different sizes.

🔵 Basic Usage

```
# terraform.workspace returns:
# "default" → when on the default workspace
# "staging" → when on staging workspace
# "prod"    → when on prod workspace

# Use in resource names
resource "aws_s3_bucket" "app" {
  bucket = "myapp-${terraform.workspace}-data"
  # dev:     "myapp-dev-data"
  # staging: "myapp-staging-data"
  # prod:    "myapp-prod-data"
}

# Use in tags
resource "aws_instance" "web" {
  tags = {
    Environment = terraform.workspace
    Name        = "web-${terraform.workspace}"
  }
}
```

🔵 Workspace-Specific Configuration Patterns

```
# Pattern 1: Lookup map (most idiomatic)
locals {
  env_config = {
    default = {
      instance_type  = "t3.micro"
      min_size       = 1
```

```
      max_size       = 2
      multi_az       = false
    }
    staging = {
      instance_type  = "t3.medium"
      min_size       = 1
      max_size       = 3
      multi_az       = false
    }
    prod = {
      instance_type  = "t3.large"
      min_size       = 3
      max_size       = 10
      multi_az       = true
    }
  }

  # Select config for current workspace, fall back to "default"
  config = lookup(local.env_config, terraform.workspace,
local.env_config["default"])
}

resource "aws_instance" "web" {
  instance_type = local.config.instance_type
  count         = local.config.min_size
}

resource "aws_db_instance" "main" {
  instance_class = local.config.instance_type
  multi_az       = local.config.multi_az
}
```

```
# Pattern 2: Conditional based on workspace
locals {
  is_prod    = terraform.workspace == "prod"
  is_dev     = terraform.workspace == "default" || terraform.workspace ==
"dev"

  # Use in resources
  db_instance_class = local.is_prod ? "db.r5.large" : "db.t3.medium"
  enable_monitoring = local.is_prod ? true : false
  backup_retention  = local.is_prod ? 30 : 1
}

resource "aws_db_instance" "main" {
  instance_class          = local.db_instance_class
  monitoring_interval     = local.enable_monitoring ? 60 : 0
  backup_retention_period = local.backup_retention
}
```

```
# Pattern 3: Workspace-specific variable files (used with -var-file flag)
# config/default.tfvars
instance_type = "t3.micro"
min_capacity  = 1

# config/prod.tfvars
instance_type = "t3.large"
min_capacity  = 5

# Apply command:
terraform apply -var-file="config/${terraform.workspace}.tfvars"
# ← Note: you can't use terraform.workspace in -var-file flag directly
# This must be done in the shell:
terraform apply -var-file="config/$(terraform workspace show).tfvars"
```

🔵 Workspace in Backend Configuration

```
# Workspace name is automatically included in the S3 key
# for non-default workspaces:

terraform {
  backend "s3" {
    bucket = "mycompany-terraform-state"
    key    = "networking/terraform.tfstate"
    # default workspace: networking/terraform.tfstate
    # staging workspace: env:/staging/networking/terraform.tfstate
    # prod workspace:    env:/prod/networking/terraform.tfstate
  }
}
```

🔵 Short Interview Answer

> "`terraform.workspace` is a built-in expression that returns the current workspace name. It's used to make resources workspace-aware — differentiating resource names, selecting different instance sizes, or enabling/disabling features per environment. The most robust pattern is a local lookup map keyed by workspace name that returns a full configuration object, with a fallback to a default config for unknown workspaces. This avoids scattered `terraform.workspace ==` conditionals throughout the code and keeps environment-specific config in one place."

Topic 46: ➕ ⚠️ Workspace Anti-Patterns & Limitations

🔵 What It Is (Simple Terms)

Workspaces are frequently misused in ways that create operational risk and maintenance burden. Understanding these anti-patterns helps you both avoid them and explain to interviewers why you'd choose separate state files over workspaces for production.

🔵 Anti-Pattern 1: Production Environments in OSS Workspaces

```
The risk scenario:

# Engineer Alice is doing urgent prod fix
terraform workspace select prod    ← selected correctly
terraform plan                     ← looks fine
terraform apply                    ← good

# Five minutes later, same terminal, different task:
terraform workspace select dev     ← switches to dev
# ... writes some dev changes ...
terraform apply                    ← applies dev changes to dev state ✓

# Next day, same terminal, forgot they switched yesterday:
terraform plan                     ← on dev workspace still
# Looks fine (smaller dev config)
terraform apply -var-file=prod.tfvars ← applies prod values to DEV state!

# There is NO guard, NO warning, NO enforcement
# The workspace name is just a label — it doesn't enforce anything
```

🔵 Anti-Pattern 2: Different Resources Per Workspace via Conditionals

```
# ❌ This gets unmaintainable very quickly
resource "aws_waf_web_acl" "main" {
  count = terraform.workspace == "prod" ? 1 : 0  # only in prod
  # ...
}

resource "aws_shield_protection" "main" {
  count = terraform.workspace == "prod" ? 1 : 0  # only in prod
}

resource "aws_cloudwatch_dashboard" "main" {
  count = contains(["staging", "prod"], terraform.workspace) ? 1 : 0
}

resource "aws_backup_plan" "db" {
  count = terraform.workspace == "prod" ? 1 : 0
}

# After 6 months: dozens of count = workspace == "prod" conditionals
```

```
# Config is hard to read, understand, and debug
# Better approach: separate directory for prod with all resources defined
```

🔵 Anti-Pattern 3: Assuming Workspace = Environment Isolation

```
❌ WRONG mental model:
  "Each workspace is its own isolated environment"
  "If I apply to staging workspace, prod is safe"

✅ CORRECT mental model:
  "Workspace is just a different state file"
  "The SAME code runs against DIFFERENT state"
  "Nothing prevents running prod code against dev state or vice versa"

Real isolation requires:
  → Separate AWS accounts (the gold standard)
  → Separate IAM roles with account-scoped permissions
  → Physical directory separation
  → Workspaces alone provide ZERO security isolation
```

🔵 Anti-Pattern 4: Using `terraform.workspace` for Security Decisions

```
# ❌ NEVER use workspace name as a security boundary
resource "aws_s3_bucket_policy" "data" {
  policy = jsonencode({
    Statement = [
      {
        Effect = terraform.workspace == "prod" ? "Deny" : "Allow"
        Principal = "*"
        Action = "s3:DeleteObject"
      }
    ]
  })
}

# Problem: workspace name is just a string
# Someone can create "prod" workspace in their personal account
# Or switch to "prod" workspace in dev and apply — accidentally makes dev
bucket
# as restrictive as prod, or makes prod bucket as permissive as dev

# Security policies should be hardcoded or come from verified variables,
# never from workspace name alone
```

🔵 Anti-Pattern 5: Large Numbers of Workspaces

```
❌ Creating one workspace per feature branch or PR in a large team:
  workspace "feature-auth-redesign"
  workspace "feature-new-db-schema"
  workspace "feature-api-v2"
  workspace "fix-security-group-rules"
  workspace "hotfix-cert-rotation"
  ... 50 workspaces

Problems:
  → terraform workspace list becomes unusable
  → Old workspaces accumulate (people forget to destroy)
  → Orphaned cloud resources from abandoned feature branches
  → No auto-cleanup mechanism in OSS workspaces
  → State files pile up in S3

Better: Use Terraform Cloud with ephemeral workspaces and auto-destroy
Or: Use PR-based Terraform pipelines that create/destroy their own state
```

🔵 The Workspace Limitation Summary

```
OSS Workspace Limitations:
  ❌ No per-workspace variables in CLI
     (must use -var-file, environment variables, or workspace lookup maps)

  ❌ No per-workspace access control
     (whoever can run terraform in the directory can apply any workspace)

  ❌ No enforcement of which config runs against which workspace
     (purely convention — easily violated)

  ❌ S3 key path changes for non-default workspaces
     (env:/<name>/ prefix) — can be confusing for state path management

  ❌ No workspace-level run history or audit log
     (who applied what to which workspace, when?)

  ❌ No approval gates per workspace
     (prod shouldn't be auto-applied but OSS workspaces can't enforce
this)

  ❌ Destroying default workspace is not allowed
     (must always have at least the default workspace)
```

🔵 When Workspaces ARE the Right Tool

```
✅ Ephemeral environments with identical config
   (PR environments, feature branch testing)

✅ Single-user or small team with strong discipline
   (one person manages all environments, low risk of mistake)

✅ Simple proof-of-concept or development work
   (not production-critical)

✅ When paired with Terraform Cloud workspaces
   (TFC workspaces have per-workspace variables, access control,
   and run history — they solve most OSS workspace limitations)
```

🔵 Short Interview Answer

> "The main workspace anti-patterns are: using OSS workspaces for production environments where the risk of selecting the wrong workspace and applying to wrong state is too high; stuffing environment differences into `terraform.workspace` conditionals that make the config unmaintainable; assuming workspaces provide environment isolation — they don't, workspace is just a state file label with no security enforcement; and using workspace name for security decisions. The fundamental limitation of OSS workspaces is that nothing enforces which code runs against which workspace. For persistent production environments, separate directories with separate state files provide physical separation, team-based access control, and per-environment configuration that workspaces can't match."

🔵 Common Interview Questions

Q: "We use Terraform workspaces for dev/staging/prod. Is that a good pattern?"

> "For ephemeral environments it's fine — workspaces are great for spinning up temporary test environments from the same config. For persistent dev/staging/prod environments, I'd push back on this pattern. The main risk is there's no enforcement of which config runs against which workspace — a `terraform workspace select prod` followed by an accidental apply with staging variables (or vice versa) can have serious consequences. I'd recommend separate directories for each persistent environment, backed by separate AWS accounts and IAM roles. This provides true environment isolation, per-environment access control, and prevents cross-environment accidents at the filesystem level."

📊 Category 6 Summary — Quick Reference Card

Topic	One-Line Summary	Interview Weight

Topic	One-Line Summary	Interview Weight
35. What state is	JSON memory mapping config → cloud resources + all attributes	⭐⭐⭐⭐⭐
36. Remote backends ⚠	S3+DynamoDB, GCS, TFC — locking, versioning, encryption setup	⭐⭐⭐⭐⭐
37. State locking	DynamoDB conditional write → lock record, stuck lock scenarios	⭐⭐⭐⭐
38. state commands ⚠	mv (rename), rm (unmanage), list, show, pull (backup!), push	⭐⭐⭐⭐⭐
39. Splitting state ⚠	Split by domain, state mv across files, SSM for cross-stack refs	⭐⭐⭐⭐⭐
40. Drift detection	refresh, refresh-only, -detailed-exitcode for CI/CD alerting	⭐⭐⭐⭐
41. State security	Plaintext secrets in state, KMS encryption, IAM separation	⭐⭐⭐⭐⭐
42. force-unlock	Verify no apply running first — DynamoDB backup removal	⭐⭐⭐⭐
43. Workspaces	OSS = multiple states from one config; TFC = full isolation	⭐⭐⭐⭐
44. Workspaces vs dirs ⚠	Dirs for prod (isolation), workspaces for ephemeral	⭐⭐⭐⭐⭐
45. terraform.workspace	Lookup map pattern for workspace-specific config	⭐⭐⭐
46. Workspace anti-patterns ⚠	Prod in workspaces, workspace ≠ isolation, conditional sprawl	⭐⭐⭐⭐

🔑 Category 6 — Critical Rules

```
State:
  State = Terraform's memory, NOT infrastructure backup
  ALL attributes stored in plaintext — passwords, keys, tokens
  Never delete state — resources become orphaned
  serial increments on every write — enables concurrent detection

Remote Backend (S3):
  S3 = storage, DynamoDB = locking (separate concerns)
  Always: versioning + SSE-KMS + public access block + HTTPS-only policy
  Backend config does NOT support variable interpolation
  State key: <env>/<component>/terraform.tfstate

State Commands golden rule:
  Always: terraform state pull > backup-$(date).tfstate FIRST
```

```
    Always: terraform plan AFTER any state manipulation
    Never: state push -force unless restoring from known-good backup

  Workspaces vs Directories:
    Workspaces = same code, different state (no isolation enforcement)
    Directories = different code, different state (physical isolation)
    Production → Separate directories + separate AWS accounts
    Ephemeral testing → Workspaces are fine

  force-unlock:
    Verify no apply is running BEFORE unlocking
    Lock ID comes from the error message
    DynamoDB delete is the nuclear option
```

🎯 Category 6 — Top 5 Interview Questions to Master

1. **"What is Terraform state and why does it exist?"** — mapping config to cloud, computing diffs, serial + lineage
2. **"How would you set up a production-grade S3 backend with all security best practices?"** — versioning, KMS, bucket policy, DynamoDB table, IAM roles per env
3. **"Explain the difference between `terraform state mv`, `rm`, `pull`, and `push` and when you'd use each"** — surgical operations, backup first, plan after
4. **"When would you use Terraform workspaces vs separate state files?"** — workspaces for ephemeral, directories for production, workspace ≠ security isolation
5. **"A colleague says sensitive = true protects their database password in state. What do you say?"** — sensitive only redacts CLI output, state is always plaintext, need KMS + IAM + minimize secrets in state

---

> **Next:** Category 7 — Modules (Topics 47–54) Type `Category 7` to continue, `quiz me` to be tested on Category 6, or `deeper` on any specific topic.